



KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY
Department of Computer Science and Engineering (CSE)
Compiler Project Report

Submitted To:

Md. Badiuzzaman Shuvo

Lecturer

Department of Computer Science and
Engineering

Khulna University of Engineering &
Technology, Khulna

Subah Nawar

Lecturer

Department of Computer Science and
Engineering

Khulna University of Engineering &
Technology, Khulna

Submitted By:

Tawhidul Hasan

Roll No: 2107004

Year: Third

Semester: Second

Department of Computer Science and
Engineering

Khulna University of Engineering &
Technology, Khulna

Date of Submission: 30/03/2026

Introduction

Arcane is a compiler project that implements a full source-to-executable workflow for a themed programming language. The project begins with lexical analysis and syntax parsing, then continues through semantic validation, intermediate code generation, optimization, C code generation, native compilation, and runtime execution.

Objectives

1. Build a reliable lexer and parser aligned to the Wizard keyword specification.
2. Implement semantic validation for declarations, types, function signatures, and control-flow correctness.
3. Generate intermediate code and an optimized intermediate form.
4. Translate intermediate representation into valid C code.
5. Compile generated C code into native executables for valid programs.
6. Provide structured negative and edge-case testing with reproducible artifacts.
7. Support demonstration through both command-line workflow and web API integration.

Project Scope

The implemented scope includes:

- Lexer rules and tokenization for Wizard64 language features.
- Bison parser grammar and parse execution flow.
- Symbol table management for variables, functions, and parameters.
- Semantic checks and categorized diagnostics.
- Intermediate code generation and optimization.
- C code emission from intermediate representation.
- Automated grading suite with logs, generated C snapshots, executable artifacts, and summary reports.
- Optional backend API to run compiler workflow from a browser interface.

Tools and Technologies Used

- Flex for lexical analyzer generation.
- Bison for parser generation.
- g++ for building parser executable and generated C programs.

- Make for building orchestration.

System Workflow

1. Wizard source file (.wiz) is parsed after lexical tokenization.
2. Semantic checks validate declarations, types, conditions, and function calls.
3. Intermediate code is produced in raw form.
4. Optimizer transforms the intermediate code into an optimized form.
5. Code generator emits output/output.c.
6. g++ compiles generated C into a native executable.
7. Executable produces runtime output for valid programs.

Implementation Overview

Lexer Module

The lexer supports Wizard keywords, operators, delimiters, literals, and comment forms, with line-aware token output formatting.

Parser Module

The parser processes language structure and orchestrates later stages including semantic handling, ICG, optimization, and codegen handoff.

Semantic Layer

Semantic validation includes undeclared identifier detection, duplicate declaration checks, type mismatch checks, condition validation, and function argument arity checks.

Intermediate Code and Optimizer

The system emits intermediate code and an optimized variant to make transformation effects inspectable and reportable.

Code Generator

Code generation converts IR instructions into C statements, including function emission, call lowering, conditional/goto mapping, print handling, and main-body emission.

Testing and Validation Strategy

1. Valid tests: confirm parse success, C compilation success, and correct runtime behavior.
2. Negative tests: verify lexical, syntax, and semantic diagnostics.
3. Edge tests: evaluate boundary behavior and robustness.

Unique and Exceptional Features

The following features are notable strengths of this project:

1. Wizard language identity with a strict implementation contract.

- The language design is specification-led, not only theme-led. The keyword contract, grammar, and samples are aligned to the same source of truth.
- This improves consistency between documentation, tokenization behavior, and parser expectations.

2. Full traceability across every compiler stage.

- The project makes each step visible: source program, parser behavior, semantic diagnostics, IR, optimized IR, generated C, and runtime output.
- This level of traceability is especially valuable for academic defense and debugging.

3. Dual IR observability (RAW and OPT forms).

- Both unoptimized and optimized intermediate code are explicitly exposed.
- This enables direct verification of optimization impact instead of relying on assumptions.

4. Structured diagnostic categorization.

- Lexical, syntax, and semantic signals are distinguishable in automated summaries.
- This supports objective rubric mapping and rapid failure triage.

5. Maintainable code generation structure.

- Codegen responsibilities are separated into clear units such as line classification, pending argument buffering, function emission, and main emission.
- This modular approach supports onboarding, review, and extension.

6. Practical backend safety architecture.

- Browser-driven compilation is guarded by input-size limits, per-process timeouts,

no-shell process spawning, temporary workspace isolation, and serialized execution.

- These controls reduce risk and improve runtime stability for repeated interactive use.

7. Advanced lexical behavior for a student compiler.

- The lexer supports nested multi-line comments in addition to single-line comments.
- Nested comment handling is a non-trivial lexer capability that strengthens language robustness.

8. Constant Folding and Optimization Evidence.

Optimization behavior is observable in this project rather than hidden.

- Constant folding is implemented as a dedicated pass in the optimizer.
- Algebraic simplification, copy propagation, and dead code elimination are also applied in iterative optimization rounds.
- The project includes optimizer-focused tests, to verify these transformations.
- Report evidence can show explicit before/after examples such as $t1=5+3$ to $t1=8$.

Edge Cases Handled

The grading suite explicitly covers edge-case behavior and records evidence per case.

1. Empty program boundary.

- Handling: accepted as minimal valid structure and can complete pipeline execution.

2. Nested control structures.

- Handling: nested loop/condition flow is parsed and lowered with stable branch labels.

3. Multiple errors in one input.

- Handling: semantic diagnostics are captured for review rather than only first-failure visibility.

4. Loop-control misuse outside loops.

- Handling: semantic errors are raised for `BREAK_SPELL` and `KEEP_CASTING` used outside valid loop context.

5. Function call before declaration.

- Handling: semantic undeclared-function validation prevents invalid call lowering.

Conclusion

Arcane demonstrates a complete and demonstrable compiler pipeline with reproducible evidence generation. The project meets its core educational and engineering objectives by combining language processing modules, robust test categorization, output traceability, and practical deployment support.

The current implementation is suitable for formal demonstration, grading submission, and iterative extension toward a more production-grade compiler framework.